私が企画したICTが
世界を変えるとしたら？

SW開発/HW製作 設計書

プロジェクト名：駐車場空間が狭いヴィラのための人工知能管理アプリケーション「スマートパーキング」

2024. 10. 16

（チーム名）駐車の達人

遂行段階別の主要成果物

段階	成果物	一般 ・モバイルアプリ ・Webなど	応用ソフトウェア ・ビッグデータ ・人工知能 ・ブロックチェーンなど	応用ハードウェア ・IoT ・ロボット ・ドローンなど
環境分析	市場/技術環境分析書	△	△	△
	アンケート結果書	△	△	△
	インタビュー結果書	△	△	△
要求事項分析	要求事項定義書	○	○	○
	ユースケース定義書	▲	▲	▲
アーキテクチャ設計	サービス構成図（システム構成図）	○	○	○
	サービスフロー図（データフロー図）	△	○	△
	UI/UX定義書	△	△	△
	ハードウェア/センサー構成図	-	-	○
機能設計	メニュー構成図	○	○	○
	画面設計書	○	○	△
	エンティティ関係図	○	○	△
	機能処理図（機能フロー図）	○	○	○
	アルゴリズム明細書/説明書	△	○	○
	データ収集処理定義書	-	○	-
	ハードウェア設計図	-	-	○
開発 / 実装	プログラム一覧	○	○	○
	テーブル定義書	○	○	△
	主要ソースコード	○	○	○

※ ○必須、△選択

1. 深刻化する駐車問題

살인마 만드는 '주차 전쟁'…"알아서 해결해라" 정부는 나몰라라

전영욱 기자 cinema@mk.co.kr
입력 : 2023-11-03 19:24:31

가 🖨 < 🔖

늘어난 자동차수에 비해
충분하지 않은 주차공간
주차관련 민원 빗발쳐도
사적 자치영역이라며 '난색'



서울 강남구 압구정초등학교 인근에서 강남구청 주차단속차량이 이동하고 있다. [서은 출처=연합뉴스]

주차 갈등은 하루이틀 일이 아니지만 최근 그 양상이 극에 달하고 있다. 3일 국민권익위원회에 따르면, 올해 1~9월 국민신문고에 접수된 '사유지 불법주차' 민원 건수는 6만1265건을 기록했다. 역대 최고치였던 작년 한 해 동안의 민원 건수(5만4778건)를 올해 1~3분기 만에 넘어 신기록을 경신했다. 사유지 불법주차 민원은 2020년 2만4817건, 2021년 4만44434건, 2022년 5만4778건으로 가파르게 상승 중이다.

불법주차 민원 건도 매년 급증하고 있다. 2020년 처음 300만건을 돌파한 이후 2021년 340만건, 2022년 353만건을 기록했다. 올해 1~9월은 319만건으로 이 부문 민원 건수 역시 역대 최고치를 기록할 것으로 전망된다.

주차난이 심화되는 건 주차 공간에 비해 차량 수가 많기 때문이다. 지난해 자동차 등록대 수는 2550만대에 달했다. 2013년 1940만대였으나 10년 사이에 31% 증가했다. 하지만 구축 아파트와 다가구·다세대 주택의 주차장 확보율은 낮은 편이다. 자동차 보유가 대중화가 되기 전에 주차 공간을 충분히 확보하지 않았기 때문이다. 현행법상 사유지에 대한 주차 질서 위반에 대해서는 과태료 부과, 강제 이동 등의 행정조치가 어려운 상태다.

現在、世帯当たりの車両数はアパートの場合1.1～1.6程度という統計がある。ヴィラ（多世帯住宅）の場合は平均1台以下で、増え続ける車両数に比べて駐車場の数が著しく不足している。

2. AIと映像分析市場

연평균 33% 성장하는 글로벌 AI영상분석 시장...도전하는 K-스타트업들 '주목'

전 세계 AI영상분석 시장, 2023년 181억불→2028년 753억불
메이아이, CCTV 영상처리 AI솔루션 '메쉬', 60억원 투자유치
써모아이, 열화상 카메라 분석용 AI 솔루션, 9억원 프리A 유치
클레(AI 3차원 머신비전)-트윌브랩스(영상이해 초거대 AI)도 활약

[ザ・スタック=キム・ドンジン記者]人工知能（AI）を活用した映像分析市場が急成長している。グローバル市場調査会社「モルドーインテリジェンス」によると、全世界のAI映像分析市場規模は2023年の181億1000万ドル水準から年平均33%成長し、2028年には753億5000万ドル（約99兆ウォン）の巨大市場になる見通しである。出典：ザ・スタック（The Stock）（http://www.the-stock.kr）



2. AIと映像分析市場



(사진=마켓앤마켓)

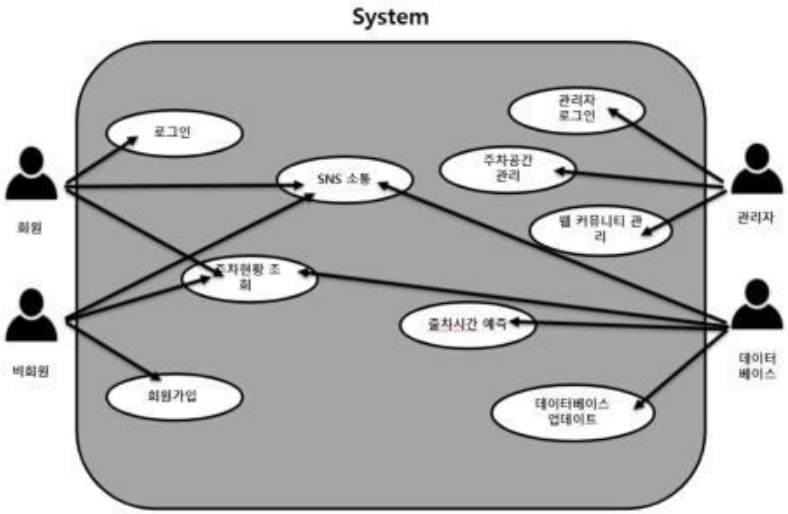
"2030년 AI 글로벌 시장 규모 1800조...2023년 9배 달할 것"

要求事項定義書

区分	機能	説明
S/W	リアルタイムのヴィラ駐車 現況ボード / 検索	ウェブカメラを通じてリアルタイムの駐車状況を確認できる。
	SNS・掲示板	住民同士が必要な情報を投稿し、コミュニケーションできる。
	映像処理	ウェブカメラで受け取った駐車状況と車両ナンバープレートを映像処理し、ラベリングできる。
	予想出庫時間の提供	過去に蓄積された各車両の駐車情報を通じて、予想出庫時間を提供できる。
	データ分析	駐車データを分析し、管理者が駐車ポリシーを改善できるインサイトを提供する。
	ユーザーインターフェース	HTML、CSS、JavaScriptを使用して直感的でレスポンシブなインターフェースを提供する。

区分	機能	説明
H/W	ウェブカメラによる車両ナンバープレート認識	ウェブカメラを用いてリアルタイムに車両ナンバープレートを認識する。
	模型車両	模型車両を用いてヴィラの駐車状況を再現する。
	車両ナンバープレート	各車両ごとに異なるナンバープレートで認識する。
	ヴィラ駐車場	指定されたヴィラ駐車場空間を通じてリアルタイムの駐車状況を提供する。
	模型ヴィラ駐車場	指定されたヴィラ駐車場空間の代わりに、デモでテストするために作られた駐車場。

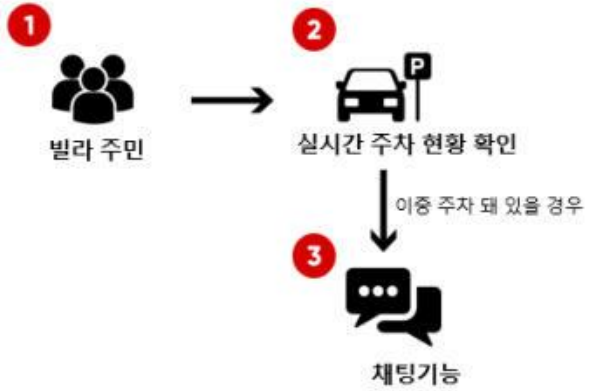
ユースケース



項目	説明	
概要	ユーザーがウェブページでリアルタイムの駐車状況と予測された出庫時間を確認できる。	
関連アクター	主アクター	ユーザー
	補助アクター	管理者、データベース
難易度	リアルタイムで駐車状況を把握し、出庫時間を予測できる技術実装能力	
事前条件	ユーザーがウェブページにアクセスできること	
事後条件	カメラがサーバーと接続されており、WiFiまたはデータが継続的に接続されていなければならない。	
シナリオ	基本シナリオ	1. ユーザーはウェブページにアクセスする。 2. ウェブページはリアルタイムの駐車状況と出庫時間予測情報をユーザーに提供する。 3. ユーザーは駐車状況と出庫時間予測情報を確認する。 4. ユーザーは必要に応じてSNSを通じて駐車スペースを予約または問い合わせる。
	代替シナリオ	A1：ウェブページにアクセスできないとき 1. ユーザーはエラーメッセージを確認する。 2. ユーザーは管理者に連絡して問題を解決する。 A2：駐車スペースがすべて満車のとき 1. ユーザーはSNSでコミュニケーションするか、別の駐車スペースを探す。

| サービス構成図 - サービス

주차 돼 있는 차를 나가야 할 때



주차를 하러 가야 할 때



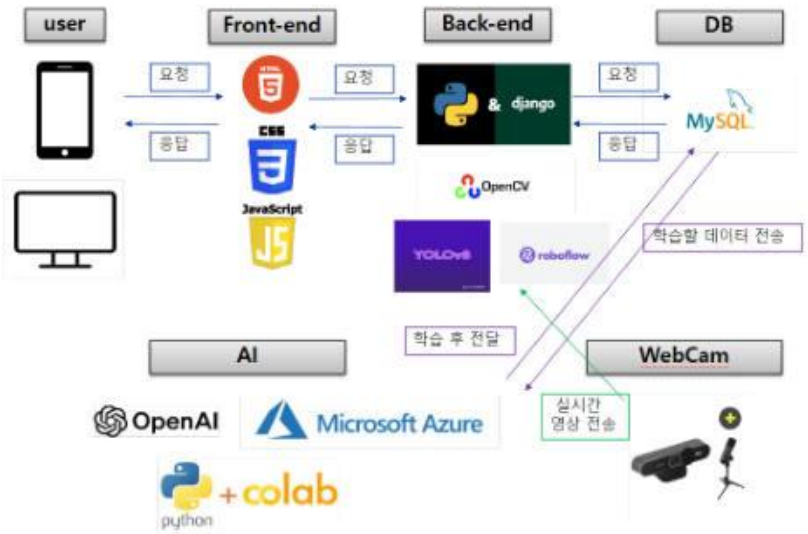
駐車中の車を出さなければならないとき

1. ヴィラの住民がウェブページにアクセスする。
2. リアルタイムの駐車状況をクリックして確認する。
3. 二重駐車がある場合、事前にチャットで連絡する。

駐車をしに行くとき

1. ヴィラの住民がウェブページにアクセスする。
2. リアルタイムの駐車状況をクリックして確認する。
3. 事前に確認後、駐車する場所がない場合は別の駐車場を利用する。

| サービス構成図 - サービスシナリオ



Front-end

機能：

1. ユーザーが駐車状況・出庫時間予測などのデータを照会できるよう要求を送信。
2. HTML、CSS、JavaScriptを用いてユーザーインターフェースを構成。
3. ユーザーの要求をBack-endへ伝達し、Back-endから受け取ったデータをユーザーに表示。

フロー：

1. ユーザー（User）が駐車状況または出庫時間予測データを要求。
2. 要求を受けてFront-endがBack-endへデータを要求。
3. Back-endからデータを受け取りユーザーに表示。

Back-end

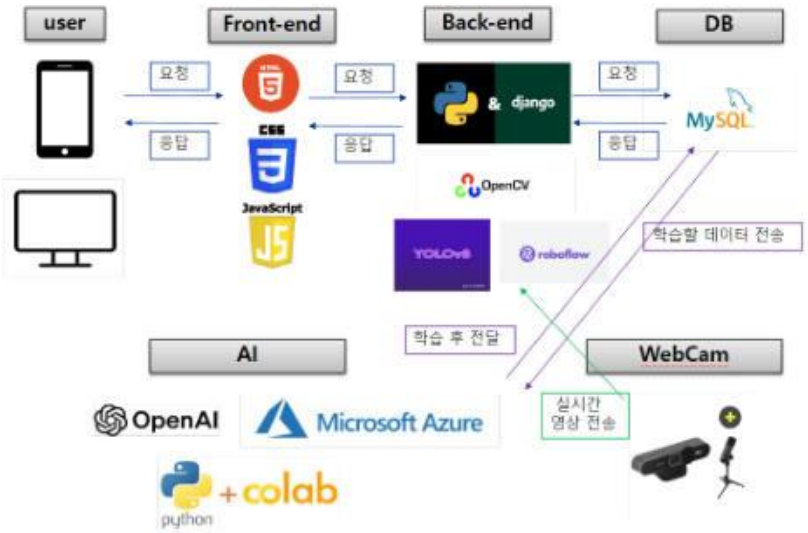
機能：

1. Front-endから受けた要求に応じてデータベース（DB）と相互作用。
2. OpenCVとTesseract OCRを用いて駐車状況データを処理。
3. Djangoフレームワークを使用してAPIを提供。
4. AIモデルを通じて学習されたデータを基に予測結果を提供。

フロー：

1. Front-endからデータを要求される。
2. 要求に応じてデータベース（DB）から必要なデータを要求。
3. OpenCVとTesseract OCRを用いてリアルタイム映像データを処理。
4. AIモデルを通じて出庫時間を予測。
5. 予測されたデータをFront-endへ伝達。

| サービス構成図 - サービスシナリオ



DB

機能：

1. 駐車状況・出庫時間・ユーザー情報などのデータを保存。
2. Back-endからデータ要求が来ると適切なデータを伝達。

フロー：

1. Back-endからデータ要求を受ける。
2. 保存されている適切なデータをBack-endへ伝達。
3. AIが駐車状況データを学習できるようデータを伝達。
4. AIから要約されたデータを保存。

AI

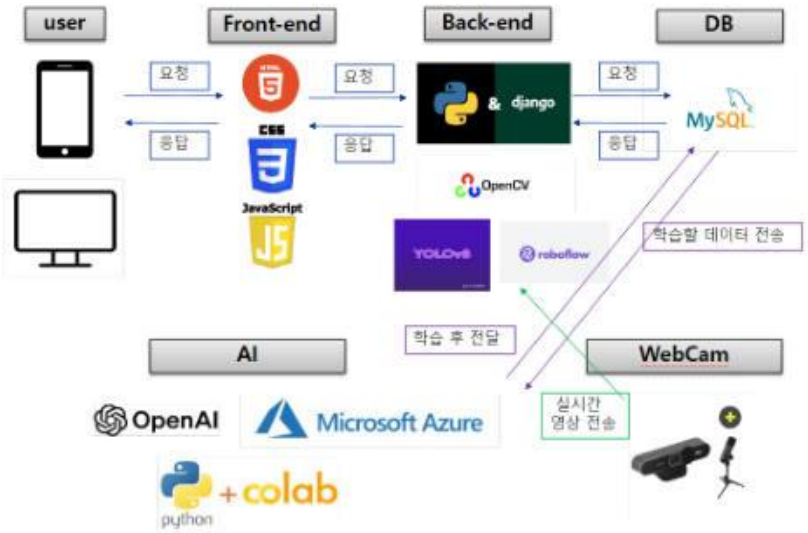
機能：

1. DBから駐車状況データを受け取り、GPRモデルを通じて学習。
2. 出庫時間を予測し、結果をDBに保存。
3. Google ColabのようなGPUサーバーを用いて学習を実行。

フロー：

1. DBから駐車状況データを受信。
2. GPRモデルを用いてデータを学習。
3. 学習されたモデルを用いて出庫時間を予測。
4. 予測結果をDBに保存。

| サービス構成図 - サービスシナリオ



Server

機能：

1. Microsoft Azureを用いてデータベースサーバーおよびデプロイサーバーとして使用される。
2. ウェブサービスが安定的に動作できるようサーバー環境を提供。

フロー：

1. AzureサーバーでBack-endとDBが動作。
2. Google ColabでAI学習のためのGPUサーバーを提供。
3. サーバーでリアルタイムに駐車状況データを収集し送信。

WebCam

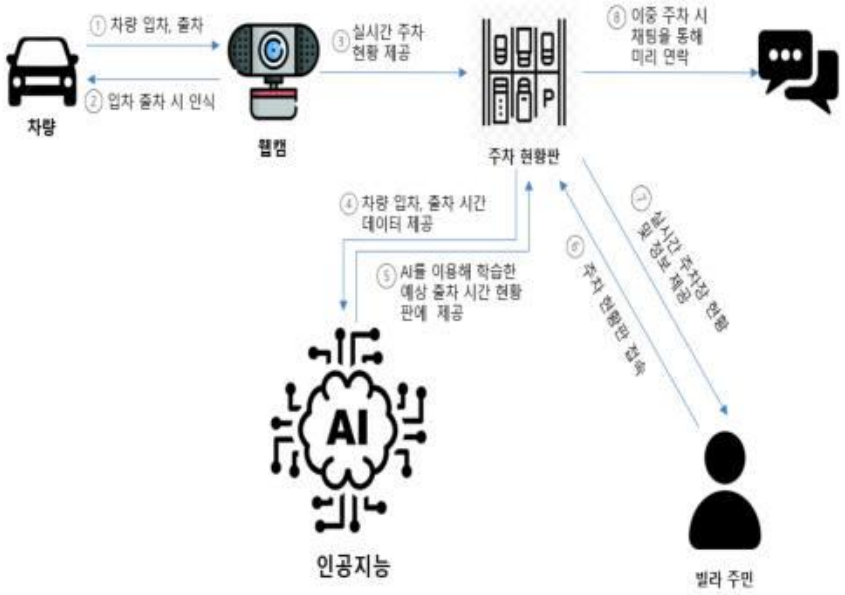
機能：

1. リアルタイムの駐車状況映像を提供してデータ収集に使用。

フロー：

1. WebCamを通じてリアルタイムに駐車状況映像を送信。
2. 送信された映像をOpenCVとTesseract OCRを用いて処理。
3. 処理されたデータをAIが学習し、予測結果を導出。

| サービスフロー図

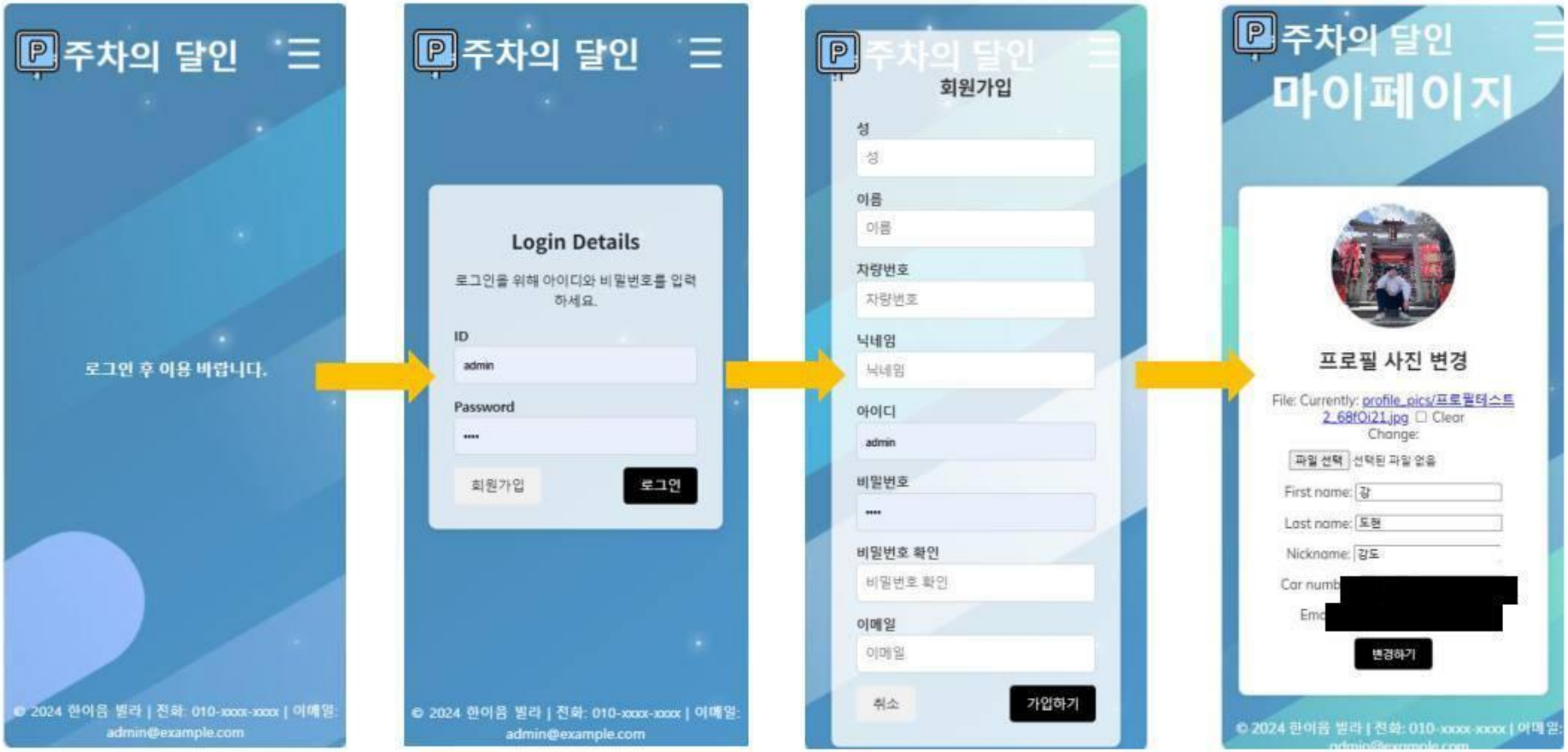


1. 車両の入庫および出庫。
2. ウェブカメラを通じて車両の入庫および出庫を認識。
3. 駐車現況ボードにリアルタイムの駐車状況を提供。
4. AIから車両の入庫・出庫データを提供。
5. 学習したデータを通じて駐車車両別の予想出庫時間を提供。
6. ヴィラ住民が駐車現況ボードにアクセス。
7. リアルタイムの駐車状況をヴィラ住民に提供。
8. 二重駐車の場合、チャットを通じて事前に連絡。

二重駐車の車主に連絡し、時間を節約できる。

UI/UX定義書 - 画面設計

- ログイン前ホームページ > 会員登録申請 > ログイン > マイページ



| UI/UX定義書 - 画面設計

- ホームページ > 各機能（駐車状況、予想出庫時間、掲示板）> 各ページ > 追加情報入力









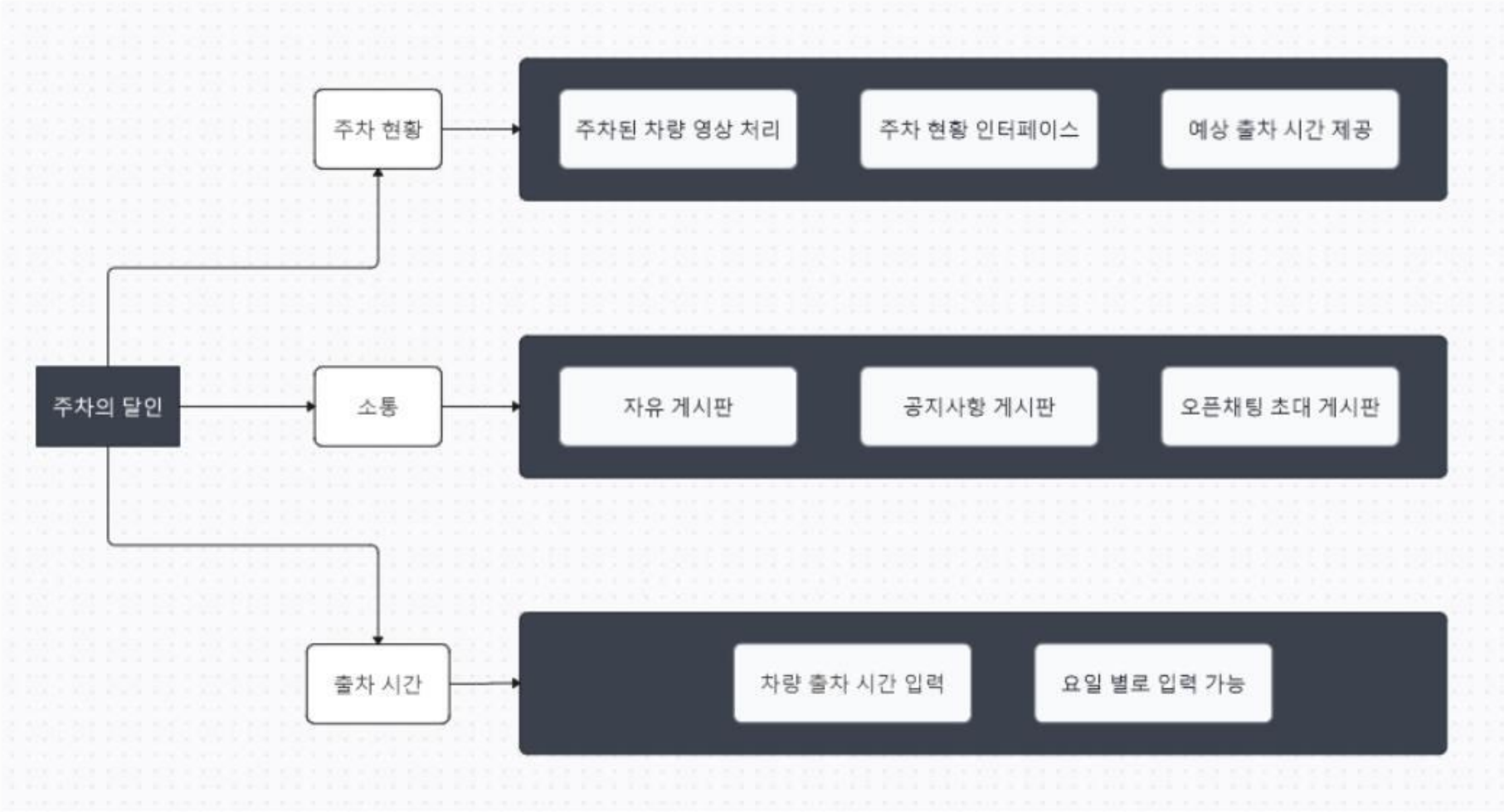


| メニュー構成図

< メニュー構成図 >



| メニュー構成図


주차의 달인
주차 현황
주차된 차량 영상 처리
주차 현황 인터페이스
예상 출차 시간 제공
소통
자유 게시판
공지사항 게시판
오픈채팅 초대 게시판
출차 시간
차량 출차 시간 입력
요일 별로 입력 가능

| 画面設計書



機能番号	LOG-01-01
機能名	ログイン前画面
機能説明	ホームページの最初の画面。ナビゲーションバーからログイン画面、ID/パスワード検索、会員登録画面へ移動できる。
処理内容	■既存会員のログイン 自分が登録したアカウントでログインできる。 ナビゲーションのログインボタンを押すとログイン画面へ移動する。 ■既存会員のID/パスワード検索 自分が登録したアカウントを検索できる。 ナビゲーションの「ID/パスワード検索」ボタンを押して新規会員の会員登録画面へ移動する。 ■新規会員の会員登録 本人の情報とメールを入力して会員登録ができる。 ナビゲーションの「会員登録」ボタンを押して新規会員の会員登録画面へ移動する。
備考	■実装機能： ナビゲーションバー（ホーム、ログイン、会員登録、ID/パスワード検索）
要求事項名	ホームページ-ログオフ

| 画面設計書



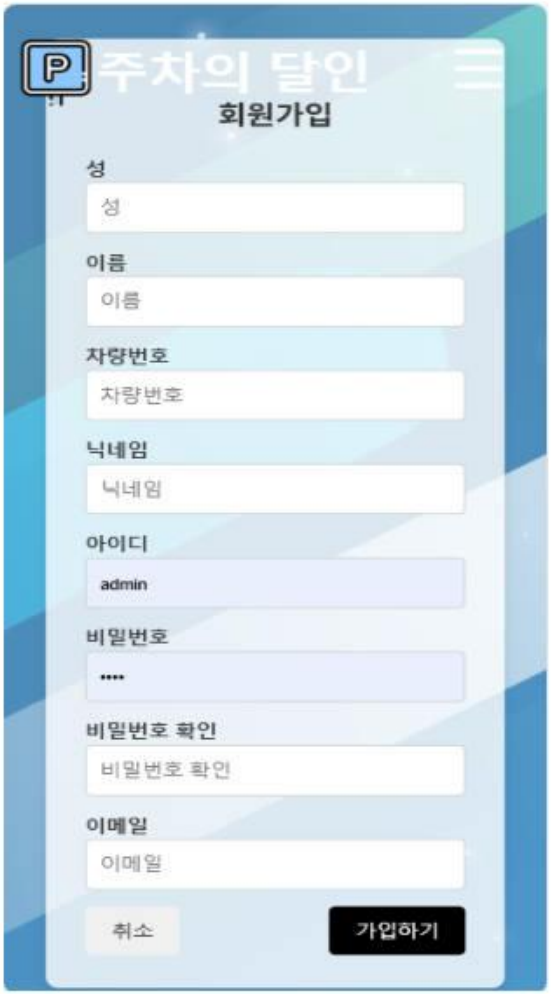
機能番号	LOG-01-02
機能名	ログイン
機能説明	ログイン画面、 既存会員は自分が登録したアカウントでログインできる。
処理内容	■ナビゲーションバー ナビゲーションバーから、Home、会員登録、ID/パスワード検索画面へ移動できる。 ■既存会員のログイン 自分が登録したアカウントでログインできる。
備考	■実装機能： ログイン、ナビゲーションバー（ホーム、会員登録、ID/パスワード検索）
要求事項名	ログイン

| 画面設計書



機能番号	LOG-01-03
機能名	ID/パスワード検索
機能説明	ID/パスワード検索画面、 既存会員は登録したアカウントを検索できる。
処理内容	■ナビゲーションバー ナビゲーションバーから、Home、会員登録、ログイン画面へ移動できる。 ■既存会員のID/パスワード検索 本人の情報とメールを入力すると、メールで認証してID検索とパスワード再設定ができる。
備考	■実装機能： メールと本人情報によるID/パスワード検索、ナビゲーションバー（ホーム、会員登録、ログイン）
要求事項名	ID/パスワード検索

画面設計書



機能番号	LOG-01-04
機能名	会員登録
機能説明	会員登録画面、 新規会員は会員登録ができる。
処理内容	■ナビゲーションバー ナビゲーションバーから、Home、ログイン、ID/パスワード検索画面へ移動できる。 ■新規会員の会員登録 本人の情報とメールを入力して会員登録ができる。
備考	■実装機能： 会員登録、ナビゲーションバー
要求事項名	会員登録

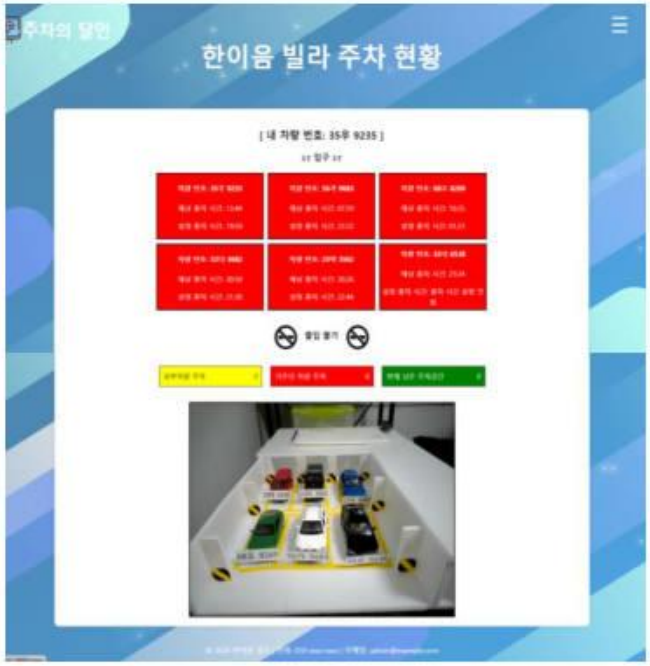
| 画面設計書





機能番号	HOME-01-01
機能名	ホーム画面
機能説明	ホームページの最初の画面、 駐車状況、出庫時間設定、掲示板画面へ移動できる。
処理内容	■駐車状況 駐車状況ページへ移動する。 ■出庫時間設定 出庫時間設定ページへ移動する。 ■掲示板 掲示板一覧ページへ移動する。
備考	■実装機能： 各ページへ移動
要求事項名	ホームページ-ログイン

| 画面設計書



機能番号	HOME-01-02
機能名	駐車状況ページ
機能説明	現在の駐車状況とGPRを利用して予想出庫時間を確認できる。空き駐車スペース：緑色、居住者駐車：赤色、外部車両駐車：黄色、空き駐車スペースのうちユーザーに推薦する駐車スペース：青色
処理内容	■駐車状況 現在の駐車状況と駐車中の車両の予想出庫時間をガウス過程回帰（GPR）で処理する。■駐車場UIの色の意味：空き駐車スペース＝緑色、居住者駐車＝赤色、外部車両駐車＝黄色、空き駐車スペースのうちユーザーに推薦する駐車スペース＝青色
備考	■実装機能： 駐車状況インターフェース、車両予想出庫時間（ガウス過程回帰, GPR）
要求事項名	駐車状況

| 画面設計書



機能番号	HOME-01-03
機能名	出庫時間設定ページ
機能説明	予想出庫時間を曜日に合わせて設定できる。
処理内容	■出庫時間設定 予想出庫時間を設定できる。 ■曜日設定 曜日を設定できる。
備考	■実装機能： 予想出庫時間設定機能
要求事項名	出庫時間設定

| 画面設計書



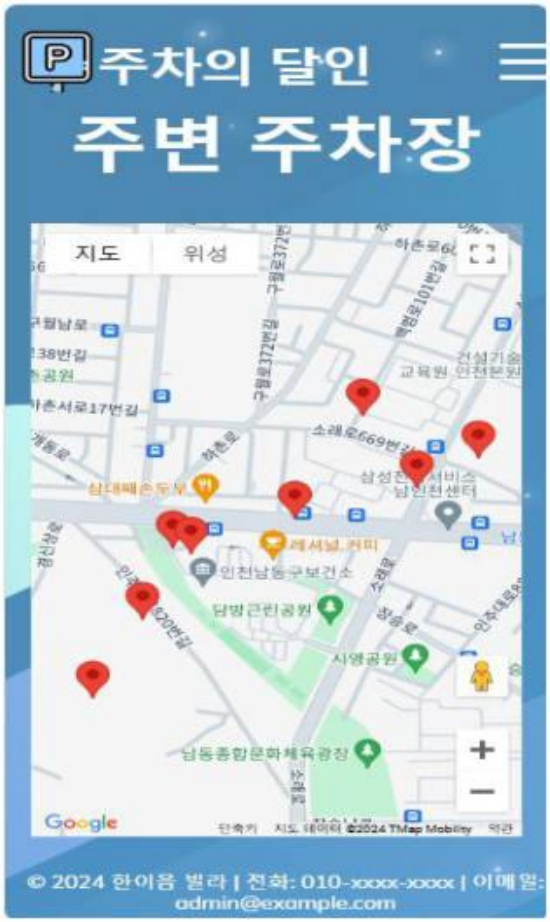
機能番号	HOME-01-04
機能名	掲示板一覧ページ
機能説明	掲示板一覧画面、 駐車状況、出庫時間設定、掲示板画面へ移動できる。
処理内容	■駐車状況 駐車状況ページへ移動する。 ■出庫時間設定 出庫時間設定ページへ移動する。 ■掲示板 掲示板一覧ページへ移動する。
備考	■実装機能： 各ページへ移動
要求事項名	掲示板一覧

画面設計書



機能番号	HOME-01-04-01
機能名	自由掲示板ページ
機能説明	自由に任意の情報を書けるページ
処理内容	■自由掲示板 掲示板一覧、投稿作成、該当の投稿へ移動、投稿検索の機能を実行する。
備考	■実装機能： 掲示板一覧、投稿作成、該当の投稿へ移動、投稿検索
要求事項名	自由掲示板一覧

| 画面設計書



機能番号	HOME-01-05
機能名	周辺駐車場の現況
機能説明	周辺駐車場の位置提供
処理内容	■周辺駐車場の提供：周辺駐車場の位置をGoogle Map APIを通じて提供。
備考	■実装機能：周辺駐車場の位置提供
要求事項名	周辺駐車場の位置提供

| 画面設計書



機能番号	HOME-01-06
機能名	プロフィール設定
機能説明	自分のプロフィール情報を入力・保存
処理内容	■プロフィール写真の更新：プロフィール写真側のDBをauth_userと1:1リレーションで紐づけて更新できる。
備考	■実装機能：プロフィール情報を管理者に提供
要求事項名	プロフィール更新

| 画面設計書 - ユーザーインターフェース（SW）

会員登録



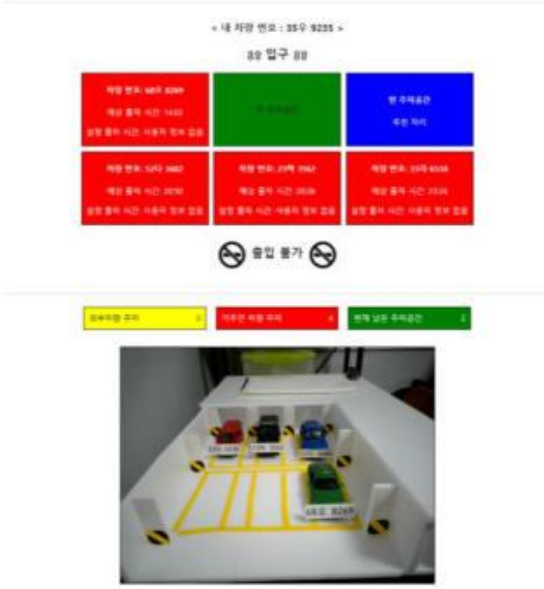
ログイン



メインページ



ハニウム ヴィラ駐車状況



| 画面設計書 - ユーザーインターフェース（SW）

メインページ



出庫時間設定



| 画面設計書 - ユーザーインターフェース（SW）

メインページ



掲示板一覧



自由掲示板



お知らせ

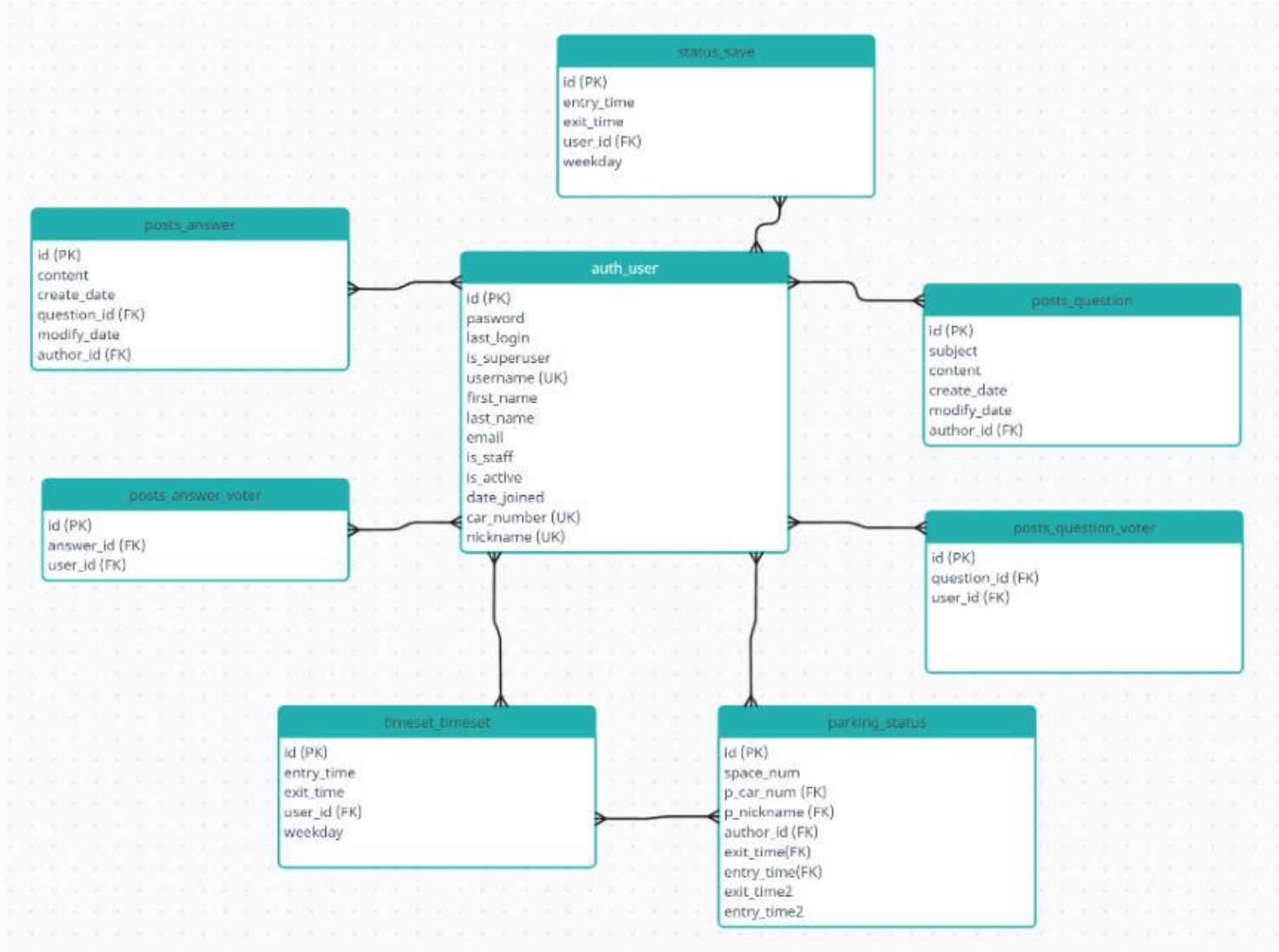


オープンチャット招待

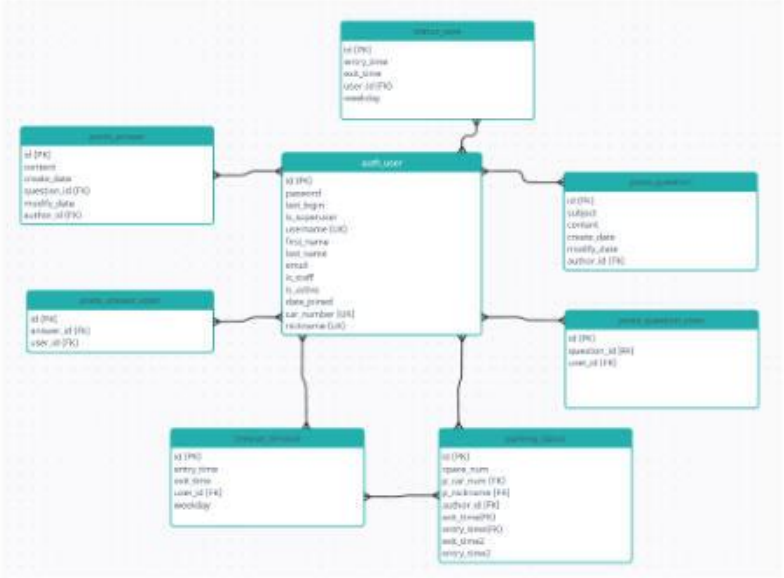


| エンティティ関係図 - ERD

追加されたDBを含めて新しく作成が必要。



| テーブル定義書 - ERD



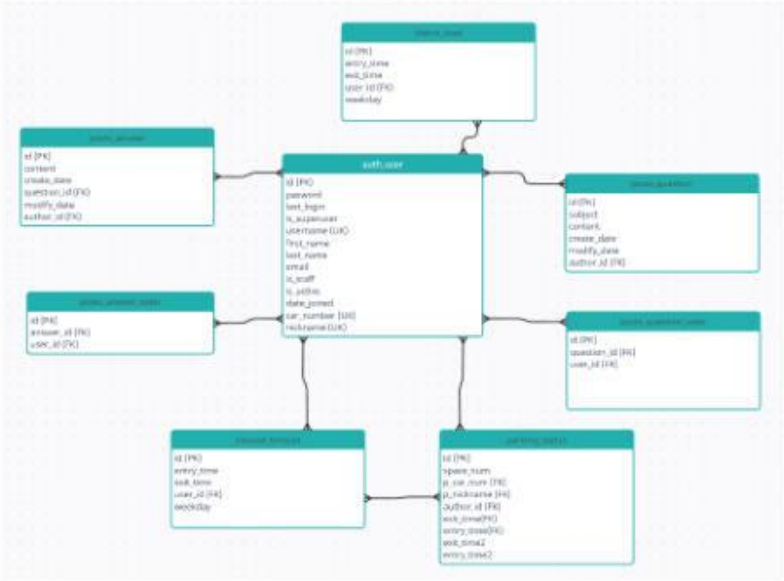
테이블 이름: auth_user

컬럼 이름	데이터 타입	제약 조건	설명
id	integer	PRIMARY KEY, AUTOINCREMENT	사용자 고유 ID
password	varchar(128)	NOT NULL	사용자 비밀번호
last_login	datetime	NULL	마지막 로그인 시간
is_superuser	bool	NOT NULL	슈퍼유저 여부
username	varchar(150)	UNIQUE, NOT NULL	사용자 이름
first_name	varchar(150)	NULL	사용자 이름
last_name	varchar(150)	NULL	사용자 성
email	varchar(254)	NULL	이메일 주소
is_staff	bool	NOT NULL	스태프 여부
is_active	bool	NOT NULL	활성화 여부
date_joined	datetime	NOT NULL	가입일
car_number	varchar(150)	UNIQUE, NULL	차량차 번호
nickname	varchar(150)	UNIQUE, NULL	사용자 닉네임

테이블 이름: parking_status

컬럼 이름	데이터 타입	제약 조건	설명
id	integer	PRIMARY KEY, AUTOINCREMENT	고유 ID
space_num	integer	NOT NULL	주차 공간 번호
p_car_num	varchar(150)	NOT NULL	주차된 차량 번호
p_nickname	varchar(150)	NOT NULL	주차된 차량 닉네임
author_id	integer	NOT NULL	작성자 ID
FOREIGN KEY (author_id) REFERENCES auth_user(id)			
FOREIGN KEY (p_nickname) REFERENCES auth_user(nickname)			
FOREIGN KEY (p_car_num) REFERENCES auth_user(car_number)			

| テーブル定義書 - ERD



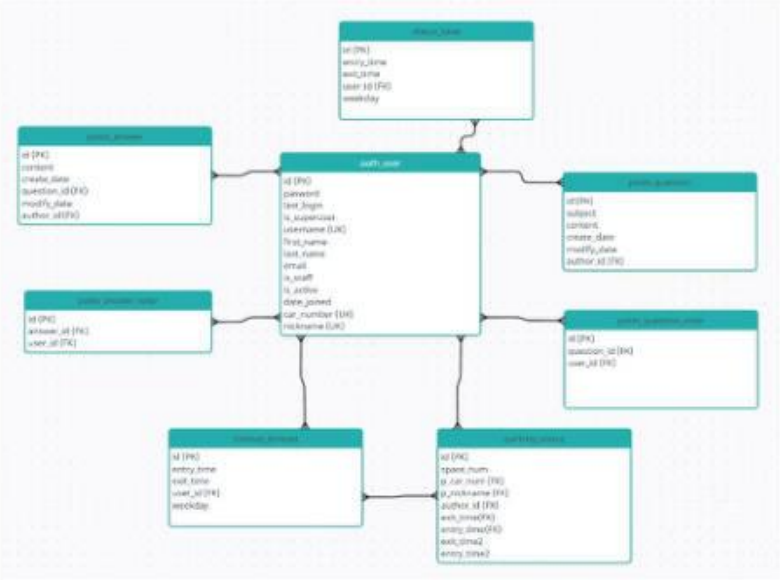
테이블 이름: posts_answer

컬럼 이름	데이터 타입	제약 조건	설명
id	integer	PRIMARY KEY, AUTOINCREMENT	고유 ID
content	text	NOT NULL	답변 내용
create_date	datetime	NOT NULL	작성일
question_id	bigint	NOT NULL	질문 ID
modify_date	datetime	NULL	수정일
author_id	integer	NOT NULL	작성자 ID
FOREIGN KEY (question_id) REFERENCES posts_question(id)			
FOREIGN KEY (author_id) REFERENCES auth_user(id)			

테이블 이름: posts_answer_voter

컬럼 이름	데이터 타입	제약 조건	설명
id	integer	PRIMARY KEY, AUTOINCREMENT	고유 ID
answer_id	bigint	NOT NULL	답변 ID
user_id	integer	NOT NULL	사용자 ID
FOREIGN KEY (answer_id) REFERENCES posts_answer(id)			
FOREIGN KEY (user_id) REFERENCES auth_user(id)			

| テーブル定義書 - ERD



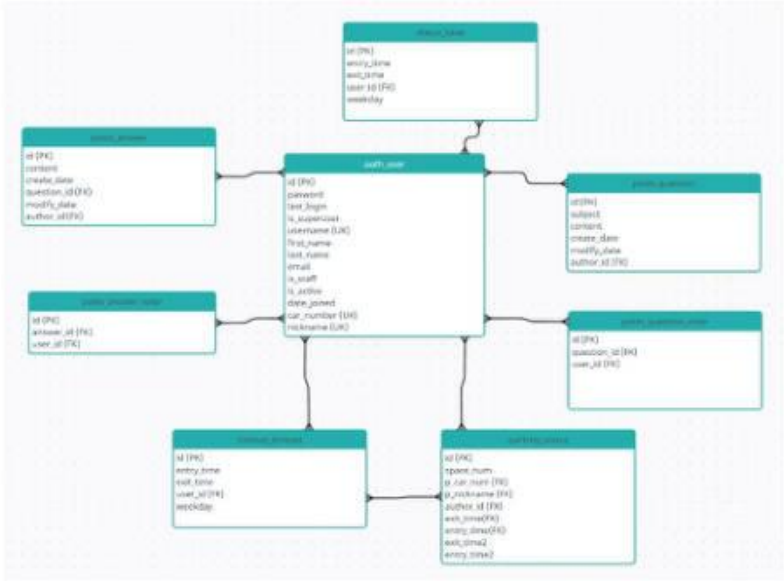
테이블 이름: posts_question

컬럼 이름	데이터 타입	제약 조건	설명
id	integer	PRIMARY KEY, AUTOINCREMENT	고유 ID
subject	varchar(200)	NOT NULL	질문 제목
content	text	NOT NULL	질문 내용
create_date	datetime	NOT NULL	작성일
modify_date	datetime	NULL	수정일
author_id	integer	NOT NULL	작성자 ID
FOREIGN KEY (author_id) REFERENCES auth_user(id)			

테이블 이름: posts_question_voter

컬럼 이름	데이터 타입	제약 조건	설명
id	integer	PRIMARY KEY, AUTOINCREMENT	고유 ID
question_id	bigint	NOT NULL	질문 ID
user_id	integer	NOT NULL	사용자 ID
FOREIGN KEY (question_id) REFERENCES posts_question(id)			
FOREIGN KEY (user_id) REFERENCES auth_user(id)			

| テーブル定義書 - ERD



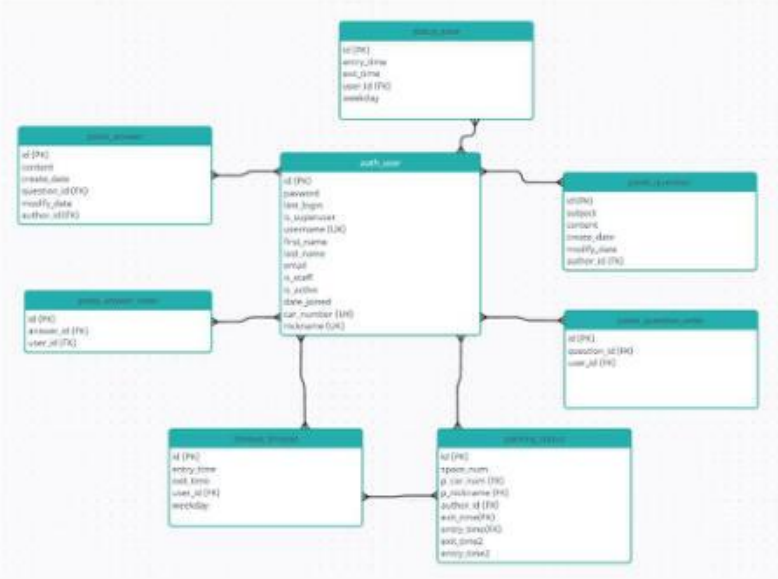
테이블 이름: timeset_timeset

컬럼 이름	데이터 타입	제약 조건	설명
id	integer	PRIMARY KEY, AUTOINCREMENT	고유 ID
entry_time	time	NOT NULL	입차 시간
exit_time	time	NOT NULL	출차 시간
user_id	integer	NOT NULL	사용자 ID
weekday	varchar(10)	NOT NULL	요일
FOREIGN KEY (user_id) REFERENCES auth_user(id)			

테이블 이름: status_save

컬럼 이름	데이터 타입	제약 조건	설명
id	integer	PRIMARY KEY, AUTOINCREMENT	고유 ID
entry_time	time	NOT NULL	입차 시간
exit_time	time	NOT NULL	출차 시간
user_id	integer	NOT NULL	사용자 ID
weekday	varchar(10)	NOT NULL	요일
FOREIGN KEY (user_id) REFERENCES auth_user(id)			

| テーブル定義書 - ERD



컬럼 이름	데이터 타입	제약 조건	설명
id	integer	PRIMARY KEY, AUTOINCREMENT	고유 ID
plate_text	varchar(20)	NOT NULL	차량 번호판 텍스트
index	integer	NOT NULL	주차 공간 인덱스
entry_time	datetime	NOT NULL	입차 시간
exit_time	datetime	NULL	출차 시간

追加データベーステーブル [Parking history]

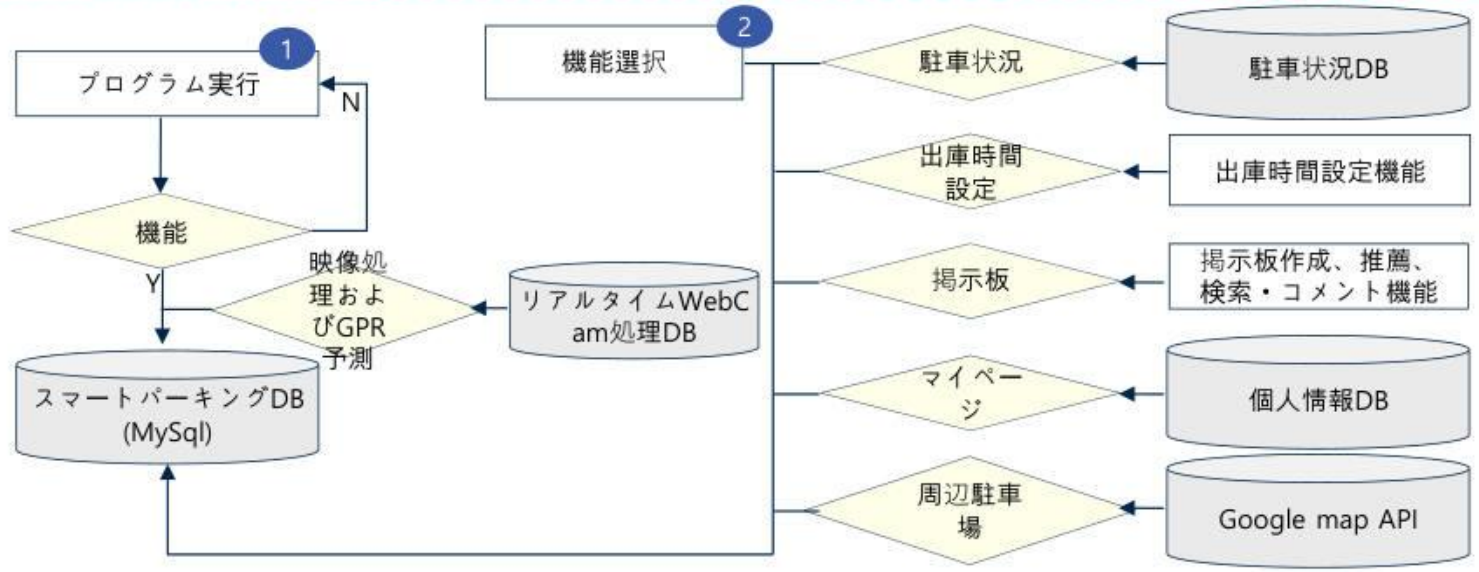
| 機能処理図（機能フロー図）

実務成果物形式

プログラムID	SMT-P01	プログラム名	スマートパーキング	作成日	2024．07．09．	Page	1/1
概要	リアルタイムカメラで受け取った駐車状況映像を処理し、ガウス過程回帰（GPR）で車両別の予想出庫時間を予測して、駐車状況とともにウェブで提供。					作成者	チーム長

機能フロー図

＜Kinect連動インタラクティブ・スマートパーキング連携図＞



1 スマートパーキングのウェブサービスを実行し、ナビゲーションバーとアイコンで機能を選択。

2 駐車状況、出庫時間設定、掲示板、マイページ、周辺駐車場などの機能を選択して実行。

| アルゴリズム明細書



シナリオ

1.Start：システムが開始されると、カメラまたはビデオファイルから映像を入力受け取る。
2.映像入力：入力された映像をフレーム単位で分割。
3.自動車検知：YOLO、Faster R-CNNなどの物体検知アルゴリズムを用いてフレーム内で自動車を検知。
4.自動車ナンバープレート検知：検知された自動車領域からナンバープレートをさらに検知。
5.ナンバープレートOCR：検知されたナンバープレート領域でTesseract OCRを用いて文字を認識。
6.駐車スペース検知：駐車スペース内の空きを検知し、現在の状態を把握。
7.結果出力：検知結果（自動車の位置、ナンバープレート情報、駐車スペース状態）を出力。
8.End：すべての処理が完了した後、システムを終了。

1. 物体検知アルゴリズム

自動車および自動車ナンバープレートOCR、駐車スペース検知

物体検知（Object Detection）：映像内の特定オブジェクトを認識し位置を追跡する技術。
この技術を自動車、自動車のナンバープレート、そして駐車スペースの検知に活用できる。

- **自動車検知：YOLO（You Only Look Once）、Faster R-CNNのようなディープラーニングベースのモデルを用いて映像内の自動車を検知する。これらのモデルは高い精度と速い速度を提供し、様々な大きさや形態の自動車を認識できる。**
- **自動車ナンバープレートOCR：ナンバープレート認識では、まずナンバープレートを検知した後、その領域を切り出してOCR（Optical Character Recognition）アルゴリズムを適用し文字を認識する。Tesseract OCRのようなオープンソースライブラリを活用でき、これによりナンバープレートのテキストを抽出する。**
- **駐車スペース検知：駐車スペースを検知するには、駐車場の領域を定義し、その領域内の空きを識別するアルゴリズムを用いる。主に映像処理技術を使用して駐車スペースの状態（空き・駐車中の車両）をリアルタイムでモニタリングする。**



＜物体検知＞

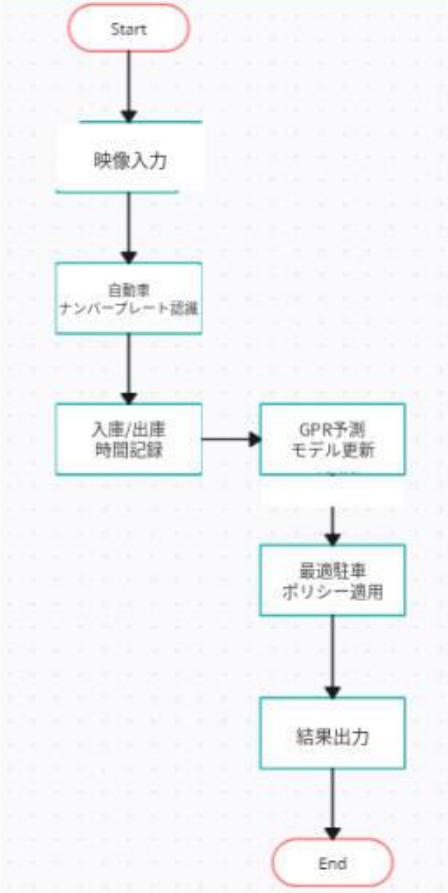


＜空き駐車スペース検知＞



＜ナンバープレートOCR＞

| アルゴリズム明細書



シナリオ

1.Start：システムが開始されると、カメラまたはビデオファイルから映像を入力受け取る。
2.映像入力：入力された映像をフレーム単位で分割。
3.自動車ナンバープレート認識：ナンバープレートOCRを通じて車両のナンバープレートを認識。
4.入庫/出庫時間記録：認識されたナンバープレート情報を基に、車両の入庫および出庫時間をデータベースに記録。
5.GPR予測モデル更新：記録されたデータを基にGPR回帰モデルを更新し、駐車スペースの効率的な配置を学習。
6.最適駐車ポリシー適用：学習された方策を基に最適な駐車スペースを推薦・割当。
7.結果出力：最適駐車スペースの割当結果を出力。
8.End：すべての処理が完了した後、システムを終了。

2. 車両の出庫・入庫時間記録およびGPR予測アルゴリズム

出庫・入庫時間記録

車両の出庫および入庫時間を記録するシステムは、駐車場の管理と運営において重要な役割を果たす。

- **システム実装：車両が駐車場に入庫または出庫する際にナンバープレートを認識し、入庫・出庫時間を自動で記録する。この記録はデータベースに保存され、後に駐車料金の計算や駐車スペースの効率的管理に活用される。**

強化学習からGPR（ガウス過程回帰）への転換

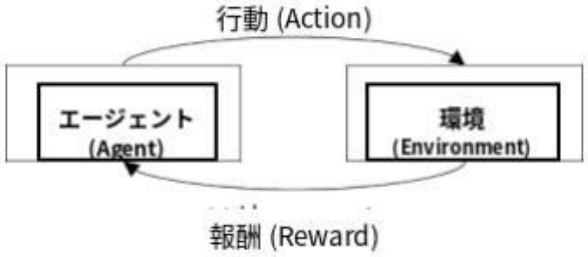
機械学習は学習方式に応じて、教師あり学習（正解ラベルで学習）、教師なし学習（データの内在構造・クラスタを学習）、強化学習（報酬信号により試行錯誤を通じて方策を学習）に分けられる。強化学習（Reinforcement Learning）は、エージェントが環境と相互作用しながら報酬を最大化する行動を学習する方法である。

転換理由：強化学習は最適方策に収束するために数万回以上の試行（環境相互作用データ）を必要とする。ヴィラ50か所ほどを回ってデータを集めたが、車両ごとの観測データが大幅に不足し、学習が収束しなかった。目的は「強化学習を使うこと」ではなく「出庫予測で待ち時間を減らすこと」であるため、少ないデータに強いGPRへ転換した。

- **GPR（Gaussian Process Regression）は回帰（教師あり学習）系のノンパラメトリックなベイズ手法である。試行錯誤なしに与えられたデータへ直接適合して予測し、予測値とともに不確実性（信頼区間）を提供するのが利点である。**
- **適用根拠：データ約100件＋不確実性情報が必要＋GPUなし ― 三つの条件すべてがGPRに適合。過去の駐車・出庫記録、時間帯、駐車頻度を入力特徴量とし、カーネル関数が類似パターンを反映して出庫時間と信頼度を併せて算出する。**

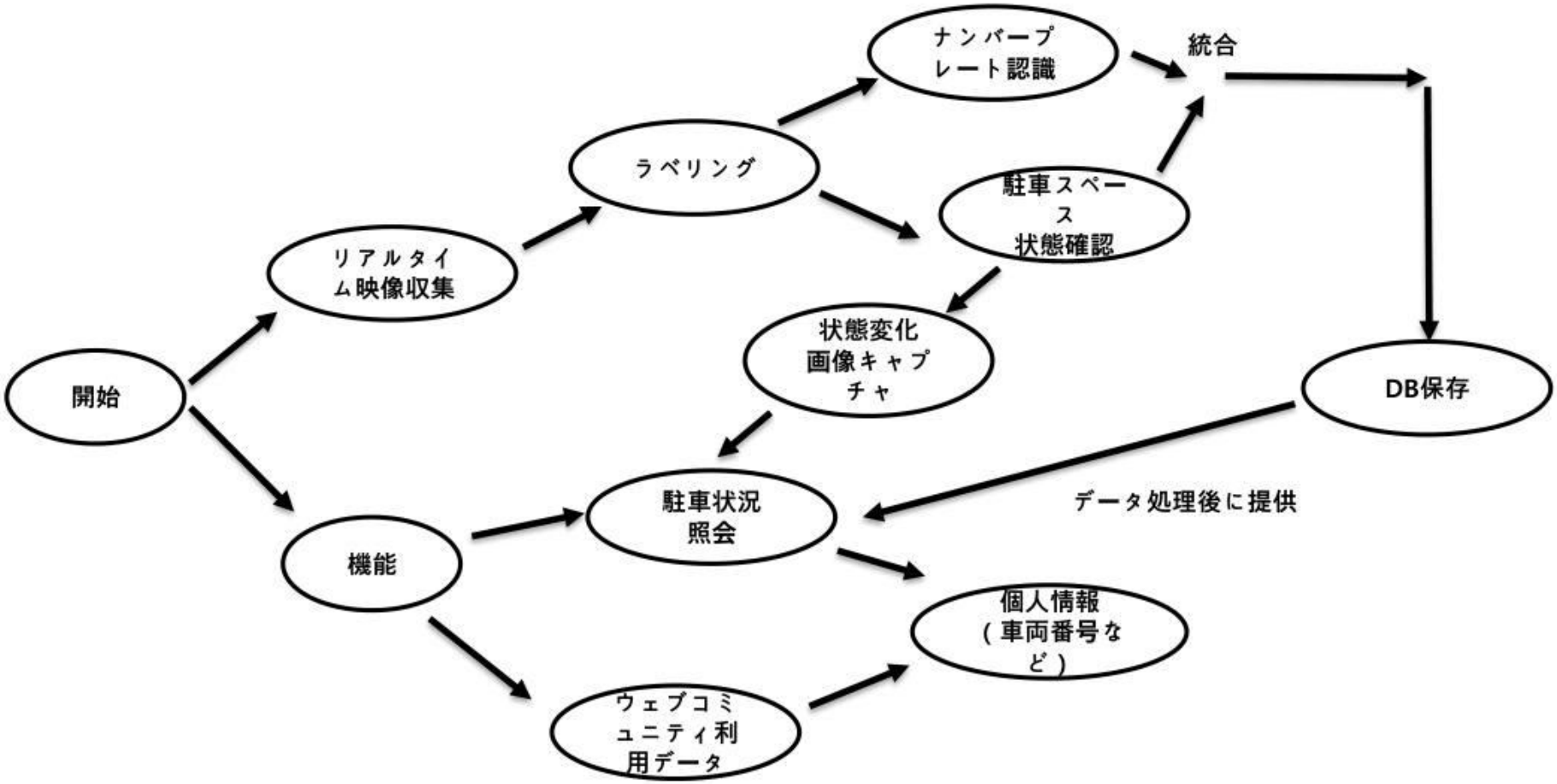


＜GPR予測パイプライン＞



＜回帰と不確実性＞

データ収集処理定義書


開始
リアルタイム映像収集
ラベリング
ナンバープレート認識
駐車スペース状態確認
統合
状態変化画像キャプチャ
DB保存
データ処理後に提供
駐車状況照会
機能
個人情報（車両番号など）
ウェブコミュニティ利用データ

プログラム - 一覧

機能分類	機能番号	機能名
LOG	LOG-01-01	ログイン前画面
	LOG-01-02	ログイン
	LOG-01-03	ID/パスワード検索
	LOG-01-04	会員登録
HOME	HOME-01-01	ホーム画面
	HOME-01-02	駐車状況ページ
	HOME-01-03	出庫時間設定ページ
	HOME-01-04	掲示板一覧ページ
	HOME-01-04-01	自由掲示板ページ
	HOME-01-04-02	お知らせページ
	HOME-01-04-03	オープンチャット招待ページ
MY	MY-01	プロフィール設定
以下省略		

| 主要ソースコード（1）

データ学習

```
!pip install roboflow

from.roboflow import Roboflow
rf = Roboflow(api_key="")
project = rf.workspace("kangdo-6svxj").project("parking-rphnx")
version = project.version(2)
dataset = version.download ("yolov8")
```

```
from ultralytics import YOLO

# YOLOV8 모델 불러오기 (예시: yolov8n.pt)
model = YOLO('yolov8n.pt')

#데이터셋 경로 설정
data_path = '/content/Parking-2/data.yaml'

#모델 훈련
model.train(data=data_path, epochs=100, imgsz=640)
```

```
from ultralytics import YOLO
from IPython.display import Image, display
import os

# YOLO 모델 불러오기
model = YOLO('/content/runs/detect/train/weights/best.pt')

# 이미지에서 예측 수행 및 결과 저장
results = model.predict(source='/content/Parking-2/train/images',
save=True, project='/content/runs', name='predict_results')

# 예측 결과가 저장된 디렉토리 경로
result_dir = '/content/runs/predict_results'

# 결과 디렉토리 내의 파일 목록 출력 및 시각화
for file_name in os.listdir(result_dir):
    if file_name.endswith(('.png', '.jpg', '.jpeg')):
        display(Image(filename=os.path.join(result_dir,file_name)))
```

| 主要ソースコード（2）

1. ウェブカメラを通じてリアルタイム映像を取得する処理

```python
import cv2

def capture_video():
    cap = cv2.VideoCapture(1)  # 웹캠 장치 번호는 1번
    if not cap.isOpened():
        print("Error: Could not open webcam.")
        return

    while True:
        ret, frame = cap.read()
        if not ret:
            print("Error: Could not read frame.")
            break

        cv2.imshow("Webcam", frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

capture_video()
```

2. 5分ごとにリアルタイム映像をキャプチャして画像として保存

```python
import cv2
import time
from datetime import datetime

def capture_images_every_5_minutes():
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        print("Error: Could not open webcam.")
        return

    while True:
        ret, frame = cap.read()
        if not ret:
            print("Error: Could not read frame.")
            break

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = f"capture_{timestamp}.jpg"
        cv2.imwrite(filename, frame)
        print(f"Saved image: {filename}")

        time.sleep(300)  # 5분 대기

    cap.release()

capture_images_every_5_minutes()
```

| 主要ソースコード（3）

3. Tesseract OCRでナンバープレート処理

```python
import cv2
import pytesseract

def process_license_plate(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)

    text = pytesseract.image_to_string(binary, config='--psm 8')
    print(f"Detected license plate: {text}")

image_path = "path_to_captured_image.jpg"
process_license_plate(image_path)
```

4. 駐車スペース状態の確認およびラベリング

```python
import cv2
import numpy as np

def detect_parking_spaces(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)

    contours, _ = cv2.findContours(binary, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

    parking_spaces = {}
    index = 1
    for contour in contours:
        area = cv2.contourArea(contour)
        if area > 500:  # 주차 공간 크기 기준
            x, y, w, h = cv2.boundingRect(contour)
            roi = img[y:y+h, x:x+w]
            space_status = "empty" if is_empty(roi) else "full"
            parking_spaces[index] = space_status
            index += 1

    print(parking_spaces)

def is_empty(roi):
    gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)
    white_pixels = np.sum(binary == 255)
    return white_pixels > (roi.size / 2)

image_path = "path_to_captured_image.jpg"
detect_parking_spaces(image_path)
```

| 主要ソースコード（4）

5. 駐車スペースとナンバープレートのラベリン
結合およびDB保存

```python
import cv2
import pytesseract
import django
from django.conf import settings
from django.db import models

# Django 설정
settings.configure(
    DATABASES={
        'default': {
            'ENGINE':
'django.db.backends.mysql',
            'NAME': 'your_db_name',
            'USER': 'your_db_user',
            'PASSWORD': 'your_db_password',
            'HOST': 'localhost',
            'PORT': '3306',
        }
    }
)
django.setup()
```

```python
class ParkingStatus(models.Model):
    space_num = models.IntegerField()
    status = models.CharField(max_length=10)
    license_plate = models.CharField(max_length=20)

def process_and_save_to_db(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)

    contours, _ = cv2.findContours(binary, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

    index = 1
    for contour in contours:
        area = cv2.contourArea(contour)
        if area > 500:  # 주차 공간 크기 기준
            x, y, w, h = cv2.boundingRect(contour)
            roi = img[y:y+h, x:x+w]
            space_status = "empty" if is_empty(roi) else "full"
            license_plate = pytesseract.image_to_string(roi, config='--psm 8').strip()

            # DB에 저장
            parking_status = ParkingStatus(space_num=index, status=space_status,
license_plate=license_plate)
            parking_status.save()
            index += 1

def is_empty(roi):
    gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)
    white_pixels = np.sum(binary == 255)
    return white_pixels > (roi.size / 2)

image_path = "path_to_captured_image.jpg"
process_and_save_to_db(image_path)
```

| 主要ソースコード (5)

Djangoモデル設定

```python
# models.py

from django.db import models

class ParkingStatus(models.Model):
    space_num = models.IntegerField()
    status = models.CharField(max_length=10)
    license_plate = models.CharField(max_length=20)
    timestamp = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Space {self.space_num}: {self.status}
({self.license_plate})"
```

Django設定ファイル修正

```python
# settings.py


DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'your_db_name',
        'USER': 'your_db_user',
        'PASSWORD': 'your_db_password',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

| 主要ソースコード（6）

駐車状態データ関数

```python
def parking_status(request):
    # 주차 상태 데이터 가져오기
    parking_spaces = ParkingSpace.objects.all()

    # 사용자별로 추천된 주차 공간 가져오기
    user = request.user
    if user.is_authenticated:
        car_number = user.car_number
        recommended_space = recommend_parking_space(car_number)
    else:
        recommended_space = None

    context = {
        'parking_spaces': parking_spaces,
        'recommended_space': recommended_space,  # 추천된 주차 공간 추가
    }

    return render(request, 'parking_status/parking_status.html', context)
```

曜日別の平均入庫・出庫時間の計算

```python
def calculate_average_times():
    """요일별 평균 입차 시간, 출차 시간, 머무르는 시간 계산"""
    days_of_week = ['월요일', '화요일', '수요일', '목요일', '금요일', '토요일', '일요일']
    average_data = {}

    for i in range(7):
        histories = ParkingHistory.objects.filter(entry_time__week_day=i + 1)  # 1: 일요일, ..., 7: 토요일

        if histories.exists():
            total_entry_time = timedelta()
            total_exit_time = timedelta()
            total_duration = timedelta()
            count = histories.count()

            for history in histories:
                entry_time_of_day = history.entry_time.time()
                exit_time_of_day = history.exit_time.time()
                duration = history.exit_time - history.entry_time

                total_entry_time += timedelta(hours=entry_time_of_day.hour,
minutes=entry_time_of_day.minute)
                total_exit_time += timedelta(hours=exit_time_of_day.hour,
minutes=exit_time_of_day.minute)
                total_duration += duration

            average_entry_time = (total_entry_time / count).total_seconds()
            average_exit_time = (total_exit_time / count).total_seconds()
            average_duration = (total_duration / count).total_seconds()

            average_data[days_of_week[i]] = {
                'average_entry_time': str(timedelta(seconds=average_entry_time)),
                'average_exit_time': str(timedelta(seconds=average_exit_time)),
                'average_duration': str(timedelta(seconds=average_duration))
            }

    return average_data
```

| 主要ソースコード（7）

予想出庫時間の計算

```
def calculate_estimated_exit_time(plate_text):
    # 예시 알고리즘: 과거의 평균 주차 시간을 사용하여 예상 출차 시간을 계산
    history = ParkingHistory.objects.filter(plate_text=plate_text).order_by('-entry_time').first()
    if history:
        average_duration =
ParkingHistory.objects.filter(plate_text=plate_text).aggregate(avg_duration=Avg(F('exit_time') -
F('entry_time')))['avg_duration']
        if average_duration:
            return history.entry_time + average_duration
    return None
```

推薦駐車スペースの

```
def recommend_parking_space(plate_text):
    # 특정 차량의 과거 주차 패턴 분석
    histories = ParkingHistory.objects.filter(plate_text=plate_text)

    # 주차 공간별로 점수를 매기기 위한 딕셔너리
    score_dict = defaultdict(int)

    for history in histories:
        # 주차된 인덱스에 대해 점수 부여
        score_dict[history.index] += 1

        # 주차된 요일에 대해 가중치 부여
        weekday = history.entry_time.weekday()
        score_dict[history.index] += weekday

        # 주차 시간의 패턴에 따라 가중치 부여 (예: 출퇴근 시간대)
        entry_hour = history.entry_time.hour
        if 7 <= entry_hour <= 9 or 17 <= entry_hour <= 19:
            score_dict[history.index] += 2

    # 모든 주차 공간의 리스트 가져오기
    all_spaces = ParkingSpace.objects.all()

    # 주차공간이 비어있지 않은 경우에도 점수를 고려하여 추천
    recommended_space = None
    max_score = -1

    for space in all_spaces:
        # 점유되지 않은 주차공간만 고려
        if not space.is_occupied:
            space_score = score_dict.get(space.index, 0)
            if space_score > max_score:
                recommended_space = space
                max_score = space_score

    # 만약 최적의 주차 공간을 찾지 못한 경우, 첫 번째 빈 주차 공간을 반환
    if recommended_space is None:
        recommended_space = all_spaces.filter(is_occupied=False).first()

    return recommended_space
```

| 参照 - S/W機能 実写写真


주차의 달인
Login Details
로그인을 위해 아이디와 비밀번호를 입력하세요.
ID
admin
Password
회원가입
로그인
© 2024 한이음 빌라 | 전화: 010-xxxx-xxxx | 이메일: admin@example.com


주차의 달인
한이음 빌라
이용 현황
현재 주차장 이용 현황을 확인해보세요.
자리가 부족할 경우 대체 주차장을 이용해보세요.


주차의 달인
주변 주차장
지도
위성
Google
© 2024 한이음 빌라 | 전화: 010-xxxx-xxxx | 이메일: admin@example.com

| 参照 - H/W機能 実写写真


1 0.95
485 2983
299 4830
34나 5678
686 3345


Full 0.95
485 2983
686 3345
290 4830
34나 5678

| 参照 - H/W機能 実写写真


Full 0.94
car 0.89
car 0.79
car 0.83
plate 0.27
plate 0.42
car 0.81
car 0.82
plate 0.49
plate 0.54
57누 1267
290아 4830

| 参照 - 開発環境および説明

区分		項目	適用内容
S/W 開発環境	ウェブ開発	Djnago 2024.1.4	ウェブサービスバックエンド開発
		Html, css	ウェブフロントエンド開発
		Windows 11 OS	コンピュータOS
	サーバーアプリケーション開発	Sqlite(5.1.73)	ユーザーおよび管理者、駐車状況、掲示板などウェブサービスに関連するデータを保存・管理するデータベース
		Apache(1.7.1)	管理者ウェブページを稼働するウェブサーバー
		サーバーOS	Linux CentOS 6.6
	映像処理	YOLO	映像処理認識（OCRおよび物体認識）
	ディープラーニング学習	Google colab	YOLO処理のためのラベリングデータ。ディープラーニングのためにGPUサーバーを使用
		Roboflow	YOLO処理のためのデータラベリング処理
H/W 構成装備	デバイス	Web cam(UHD2160)	PCに映像処理する駐車状況映像をリアルタイム提供
		PC	ソフトウェアコード処理およびウェブカメラと接続して映像処理、サーバーへJSONで送信・アップロード
	通信	USBアダプター	WebCamとコンピュータの接続
		LANアダプター	ウェブサーバー通信
	サーバー	クラウドサーバー	SW開発およびテストのためのクラウドサーバー（Microsoft Azure Server）

Thank you

